Lecture 18

Logging, Monitoring and Observability

Backend Engineering from First Principles

Detailed study notes • practical examples • interview revision

Lecture 18 — Logging, Monitoring and Observability

Backend Engineering from First Principles

> *These notes follow the supplied lecture transcript. Additional engineering cautions are explicitly marked as **Practical note** so they are not confused with the speaker's exact wording.*

1. Why these practices matter

Modern backend applications run across distributed services, servers and regions, while users may be spread around the world. Developers and operations teams need a way to understand what is happening across the application and its infrastructure.

Logging, monitoring and observability are practices implemented on a **spectrum**. There is no realistic point at which a system has implemented every possible practice or monitors every possible parameter. The goal is to collect enough useful information to operate, troubleshoot and improve the system.

They are closely related, but they answer different questions:

Practice	Main question	What it gives you
Logging	What happened?	Records of events with context
Monitoring	Is the system healthy? What is changing?	Current and historical measurements, trends and alerts
Observability	Why is the system behaving this way?	Ability to investigate internal behavior from outputs such as logs, metrics and traces

2. Logging

Logging is the practice of recording important events during the application and request lifecycle. Examples include:

- A user successfully logs in.
- A database operation is executed.
- A request fails validation.
- An unexpected error occurs.
- A security-relevant event happens.
- A business operation, such as creating a to-do, succeeds.

A useful log includes a timestamp and relevant metadata. The lecture gives examples such as user ID, request latency, function/method, and request context. Logs act like the application's journal: they help answer **what happened, when it happened, and with what context**.

2.1 Logging levels

Levels communicate the severity or importance of an event. Common levels discussed in the lecture:

Level	Meaning	Example use
`debug`	Detailed diagnostic information	Internal values and step-by-step troubleshooting during development
`info`	Normal application activity or successful business events	“To-do created”
`warn`	Something unusual or unsuccessful, but not necessarily an application fault	Failed login due to an incorrect password
`error`	An operation failed and needs attention	Database query failure or validation-related error
`fatal`	A severe condition that causes the application to stop or shut down	Unrecoverable startup/runtime failure

The lecture's example configuration uses more verbose logging in development and a less verbose level in production. Debug logs can be noisy, so production systems commonly filter them unless temporarily needed for diagnosis.

Practical note: Exact level names and fatal behavior vary by logging library. A `fatal` call does not universally terminate a process; verify the chosen library's behavior.

2.2 Human-readable vs structured logs

Unstructured / console-style logs

- Usually plain text, formatted for people.
- Convenient during local development.
- Often include colors and readable messages.
- Easy to scan in a terminal.

Structured logs

- Store each event as fields, commonly in JSON.
- Easier for log platforms to parse and search.
- Fields such as `timestamp`, `level`, `message`, `requestId`, `userId`, `route` and `statusCode` can be filtered or aggregated.
- Commonly preferred in production environments where logs are ingested by tools such as ELK or Grafana Loki.

Example human-readable log:

```
INFO request completed: GET /todos status=200 duration=18ms
```

Example structured JSON log:

```
{
"timestamp": "2026-10-30T12:00:00Z",
"level": "info",
"message": "request completed",
"method": "GET",
"route": "/todos",
"statusCode": 200,
"durationMs": 18,
"requestId": "req-123"
}
```

The key advantage of structured logging is not that JSON is more readable to a person; it is that machines can reliably extract fields.

2.3 What should be logged?

A practical event record may include:

- Timestamp and severity.
- Service/application name and environment.
- Request ID or correlation ID.
- HTTP method, route and status code.
- Duration/latency.
- Operation name and outcome.
- Relevant entity identifiers.
- Error type/message and safe diagnostic context.

Practical note — privacy and security: Do not indiscriminately log passwords, access tokens, session cookies, API keys, full payment details or sensitive personal data. Sanitize untrusted input to prevent log injection, restrict log access, and define retention policies. Prefer stable IDs over copying entire user records into logs.

3. Monitoring

Monitoring means continuously checking the health, state and performance of an application and its components. The lecture's examples include:

- CPU and memory usage.
- Requests processed per second.
- Database connection-pool state and open connections.
- Number of successful and failed requests.
- Application-specific business counts, such as to-dos created.

Monitoring produces measurements over time. The speaker describes the data as near-real-time, often with a delay of several seconds (for example, around 10–15 seconds in a conventional setup), rather than necessarily instantaneous.

3.1 Metrics

A **metric** is a numerical measurement tracked at a point in time or aggregated over a time window. Examples:

- Request rate / throughput.
- Error count and error rate.
- Average or percentile response time.
- CPU and memory consumption.
- Database connection utilization.
- Number of to-dos created.
- Number of failed business operations.

Metrics make system behavior quantifiable. They help identify trends and compare current behavior with historical behavior.

A dashboard may show:

- Average transaction time.
- Throughput.
- Errors and error rate.
- Runtime memory usage.
- Garbage-collection time.

Practical note: HTTP status codes greater than 200 should not automatically be treated as failed requests. For example, 3xx responses are redirects and many 4xx responses represent client-side outcomes. Define an error-rate policy that matches the service's semantics (often 5xx plus selected application failures), and track important 4xx categories separately.

3.2 Alerts

An alert is triggered when a monitored measurement crosses a configured condition. The lecture illustrates an error-rate threshold and a notification sent to Slack through a webhook.

A useful alert should:

1. Measure a condition that matters.
2. Have a threshold or rule.
3. Notify the responsible team through an agreed channel.
4. Provide enough context to start investigation.

Practical note: Alert thresholds should be based on service objectives and meaningful symptoms, not arbitrary numbers alone. Avoid alert fatigue by grouping, deduplicating and prioritizing alerts.

4. Observability

The lecture introduces observability as the ability to infer or determine a system's internal state by examining its external outputs. It presents three pillars:

1. **Logs** — records of individual events.

2. **Metrics** — numerical measurements and trends.
3. **Traces** — the path of a request through multiple components.

Observability is more than installing a dashboard. The application must be instrumented to emit useful data, and the infrastructure must collect, store, connect and display it.

4.1 Distributed tracing

A **trace** represents the journey of a request through the components involved in serving it. A request might pass through:

```
Client
|
Load balancer / API gateway
|
HTTP handler
|
Validation
|
Service / business logic
|
Repository
|
Database
```

A trace can show where the request originated, which components it touched, how long each segment took, and where an error occurred.

A trace is commonly made up of **spans**. Each span represents a timed operation or segment. Spans are connected to form the end-to-end trace.

The lecture's Go example creates a transaction in middleware, stores it in request context, and allows downstream service code to add attributes and events to the same transaction. This keeps work performed by different layers associated with one request.

4.2 Correlation between logs, metrics and traces

The three pillars become especially useful when connected:

- A metric or alert reveals an abnormal pattern.
- The metric helps narrow the time window, endpoint or service.
- Related logs show the events and error context.
- A trace reveals the request's path and the component/span where it failed or became slow.

This workflow turns “the API is failing” into a more actionable explanation of which operation failed and where.

5. Instrumentation and OpenTelemetry

Instrumentation is the practice of adding code or agents that measure application behavior and emit telemetry. Instrumentation can capture request duration, errors, attributes, spans and other useful details.

The lecture describes **OpenTelemetry (OTel)** as an open standard/ecosystem providing APIs, SDKs, collectors and tools for instrumenting applications across languages such as Go, Node.js and Python.

A simplified flow:

```
Backend application
|
| OTel APIs / SDK instrumentation
v
OpenTelemetry Collector (optional but common)
|
+------> Metrics backend
+------> Log backend
+------> Trace backend
|
Dashboards
and alerts
```

OpenTelemetry is not itself a dashboard or a single storage database. It provides vendor-neutral instrumentation and telemetry collection/export conventions. A backend such as New Relic, Datadog, Prometheus-compatible systems or a tracing store can receive and present the data.

6. Tooling mentioned in the lecture

Tool / platform	Role in the lecture's discussion
Prometheus	Metrics collection and querying
Grafana	Dashboards and visualization
Loki	Log aggregation and querying
Promtail	Log collection/forwarding in the Grafana stack discussed
Jaeger	Distributed tracing
OpenTelemetry	Vendor-neutral instrumentation and telemetry ecosystem
New Relic	Commercial, integrated observability platform
Datadog	Commercial observability platform
ELK stack	Log management/search ecosystem

The lecture contrasts assembling an open-source stack with adopting an integrated proprietary platform. An integrated service can reduce the operational work of maintaining multiple tools; an open-source stack can offer flexibility and control but requires configuration and maintenance.

Practical note: Tool capabilities and product names evolve. Choose tools based on required signals, scale, retention, cost, integrations, team expertise and operational burden—not popularity alone.

7. Walkthrough from the lecture's to-do backend

The demonstration uses a Go to-do application and New Relic to illustrate how code-level practices connect to dashboards.

7.1 Logger setup

- A logger is created before the application starts.

- Configuration chooses a log level based on environment.
- Development uses a more verbose/debug-friendly setup.
- Production uses JSON output so log-management tools can parse fields.

7.2 Request middleware and transaction context

- A middleware instruments incoming requests.
- It creates a transaction/trace context.
- It attaches contextual attributes such as service name, environment, IP address, user agent, request ID and user/tenant identifiers.
- It stores the transaction in request context so downstream layers can access it.

7.3 Service-level instrumentation

In the create-to-do service, the example:

1. Retrieves the transaction from context.
2. Adds relevant attributes such as user ID and to-do title.
3. Logs the start of the create operation.
4. Adds optional request attributes such as priority.
5. Performs validation and database work.
6. Records an error on the transaction and emits an error log if the database operation fails.
7. Records success details, including the created to-do ID.
8. Emits a business-event log with useful metadata.

The purpose is to associate logs and spans with the same request so that the dashboard can connect them.

7.4 Dashboard investigation

The demonstration triggers unauthorized API requests and then inspects:

- Error counts/rates.
- Related logs and their fields.
- Trace details associated with a log.
- Transaction response time and throughput.
- Runtime information such as memory and garbage-collection time.

This illustrates the relationship between instrumentation in application code and the data displayed by the observability platform.

8. A practical production workflow

```
1. Define important service and business signals
|
2. Instrument incoming requests and key operations
|
3. Emit structured logs, metrics and traces
|
4. Collect and export telemetry
|
5. Build dashboards and meaningful alerts
|
6. Investigate: alert -> metric -> log -> trace
|
7. Fix the cause and verify the improvement
```

Implementation checklist

- ■ Use consistent log levels.
- ■ Use readable console logs for local development.
- ■ Use structured logs in production.
- ■ Include timestamps, service/environment and request IDs.
- ■ Avoid logging secrets or unnecessary personal data.
- ■ Track request rate, errors, latency and resource usage.
- ■ Track business metrics that reflect actual user outcomes.
- ■ Define alert rules for actionable symptoms.
- ■ Instrument important request boundaries and dependencies.
- ■ Propagate trace context across service boundaries.
- ■ Connect logs and spans with trace/request identifiers.
- ■ Configure retention, access control and sampling where appropriate.
- ■ Test that telemetry is actually arriving in dashboards.
- ■ Ensure both developers and infrastructure/operations teams participate.

9. Common mistakes and interview-ready cautions

1. **Treating logging as observability:** Logs are one signal, not the whole picture.
2. **Collecting everything:** Excessive telemetry increases cost and noise; prioritize useful signals.
3. **Unstructured production logs:** Free-form text is harder to query consistently.
4. **No correlation IDs:** Events from one request become difficult to connect.
5. **Confusing metrics with logs:** Metrics are numerical time-series measurements; logs are event records.
6. **Assuming a dashboard creates observability:** Code must be instrumented and telemetry must be collected.
7. **Ignoring sensitive data:** Logs can become a security and privacy liability.
8. **Alerting on every anomaly:** Alerts should be actionable and tied to user/service impact.

9. **Assuming every HTTP status above 200 is an error:** Define failure semantics deliberately.
10. **Assuming traces automatically cross services:** Context propagation must be configured across service boundaries.

10. Interview questions and answers

Q1. What is logging?

Logging is recording important application events with timestamps and contextual metadata so that developers can understand what happened during execution.

Q2. What is monitoring?

Monitoring continuously measures application and infrastructure health/performance, presents current and historical trends, and can trigger alerts.

Q3. What is observability?

Observability is the ability to understand a system's internal behavior from the telemetry it emits. The lecture focuses on logs, metrics and traces as its three pillars.

Q4. What is the difference between logs and metrics?

Logs are event-oriented records with detailed context. Metrics are numerical measurements aggregated or tracked over time.

Q5. What is a distributed trace?

A trace follows one request across the services and components involved in handling it. It is composed of connected spans.

Q6. What is instrumentation?

Instrumentation adds code, SDKs or agents that measure application operations and emit telemetry such as durations, errors, attributes and spans.

Q7. What is OpenTelemetry?

OpenTelemetry is an open, vendor-neutral set of APIs, SDKs and collection conventions for generating and exporting telemetry.

Q8. Why use structured JSON logs?

They provide predictable fields that log platforms can parse, filter, index and aggregate reliably.

Q9. Why might development and production logging differ?

Development benefits from verbose, readable diagnostics. Production generally needs controlled volume and structured output for centralized processing.

Q10. How would you debug a sudden rise in API errors?

Start with the alert and metrics to identify affected service, endpoint and time window; inspect correlated logs for error context; follow traces to find the failing or slow component; then fix and verify.

Q11. What is the role of request context in tracing?

It carries the transaction/trace context through downstream functions so logs and spans from different layers can be associated with the same request.

Q12. Name tools from the lecture.

Prometheus, Grafana, Loki, Promtail, Jaeger, OpenTelemetry, New Relic, Datadog and the ELK stack.

11. Quick revision sheet

- **Logging = events / what happened.**
- **Monitoring = measurements / is it healthy and what is changing?**
- **Observability = investigate why / understand internal behavior from outputs.**
- **Three pillars = logs + metrics + traces.**
- **Logs:** timestamp, level, message, context, request ID.
- **Metrics:** request rate, error rate, latency, CPU, memory, DB connections, business counts.
- **Traces:** end-to-end request path; made of spans.
- **Instrumentation:** add measurement and telemetry emission to application code.
- **OpenTelemetry:** open, vendor-neutral APIs, SDKs and collection ecosystem.
- **Development:** human-readable console output and useful debug detail.
- **Production:** structured logs and centralized collection.
- **Investigation flow:** alert → metric → related log → trace → failing component.
- **Tools:** Prometheus (metrics), Grafana (dashboards), Loki (logs), Jaeger (traces), New Relic/Datadog (integrated platforms).

12. Scope of the supplied lecture

The lecture is an introductory, practical overview rather than an exhaustive specification. It explicitly says logging, monitoring and observability are broad topics and that implementation maturity varies across a spectrum. The demonstration focuses on a Go to-do backend, middleware-created transaction context, log levels/output formats, and a New Relic dashboard. The promotional hosting-platform segment has been omitted from these engineering notes.